

CIA-3 PROJECT REPORT

P10 – Digital Banking Account & Transaction Management System

Course: Advanced JavaScript Backend Frameworks (Node.js & Express JS)

Semester: 5th Semester

Section/Batch: <SECTION>

TEAM DETAILS

S.No	Student Name	Roll No.	Department	Section
1	<NAME>	<ROLL NO>	<DEPARTMENT>	<SECTION>
2	<NAME>	<ROLL NO>	<DEPARTMENT>	<SECTION>
3	<NAME>	<ROLL NO>	<DEPARTMENT>	<SECTION>
4	<NAME>	<ROLL NO>	<DEPARTMENT>	<SECTION>

GitHub Repository: <PASTE GITHUB REPOSITORY LINK HERE>

1. Project Overview

The project implements a simplified core-banking-style backend where customers can open accounts, complete KYC onboarding, manage beneficiaries, transfer funds and view statements. Bank staff can approve account applications, freeze accounts and monitor suspicious transactions. The backend uses Node.js, Express.js, MongoDB Atlas and Mongoose.

Objectives

- Model accounts, transactions and beneficiaries in MongoDB with sensible references and indexes.
- Implement balance validation and atomic-style fund transfer updates.
- Maintain an immutable transaction ledger and date-range account statements.
- Provide staff-facing approval and monitoring functions.
- Enforce JWT authentication, bcrypt password hashing and role-based authorization.
- Provide a GitHub-ready codebase, README and Postman collection.

2. Scope and Assumptions

- Third-party payment gateways and SMS/email integrations are outside the evaluation scope.
- Authentication is self-built using JWT; social login is not required.
- A minimal frontend is optional; Postman is used as the primary demonstration client.
- The implementation uses a single currency and timezone assumption.
- MongoDB Atlas is used for the database so the transfer workflow can use MongoDB transactions.

3. System Architecture

The request flow is: Postman/optional frontend → Express routes → validation middleware → JWT authentication/RBAC → controller business logic → Mongoose models → MongoDB Atlas. A centralized error handler converts validation, authentication, authorization, business-rule and database failures into consistent JSON responses.

Architecture Diagram

Postman / Frontend → Express Routes → Validation → JWT/RBAC → Controllers → Mongoose → MongoDB Atlas

Screenshot placeholder: Insert your final architecture diagram/screenshot here.

4. Functional Modules

#	Module	Implementation
1	Customer Onboarding & KYC Capture	Register customer, capture KYC status and create pending account.
2	Account Approval Workflow	Staff/Admin approve or reject pending account applications.
3	Account Management	View accounts, edit pending type, freeze/unfreeze, account status.
4	Beneficiary Management	Add, list and block trusted beneficiaries.
5	Fund Transfer Engine	Validated transfer with MongoDB session transaction.
6	Transaction Ledger	Every money movement creates a transaction record.
7	Account Statement Generation	Date-range statement from transaction ledger.
8	Minimum Balance & Limits	Minimum balance and daily transfer limit enforcement.
9	Suspicious Transaction Flagging	Transactions at/above threshold are flagged for review.
10	Account Freeze/Unfreeze	Staff/Admin can freeze or restore account status.
11	Interest Calculation Job Logic	Daily savings interest calculation and ledger entry.
12	Staff Monitoring Dashboard	Pending approvals, flagged transactions and frozen accounts.
13	Role-Based Access Control	Customer, bank_staff and admin permissions.

5. Database Schema

The database follows the collections proposed in the project brief: users, accounts, beneficiaries, transactions and approvals. References are used where records have independent lifecycles and are queried independently.

Collection	Key fields / relationship
users	name, email, passwordHash, role, kycStatus
accounts	userId → users, accountNumber, type, balance, status
beneficiaries	accountId → accounts, beneficiaryAccountNumber, nickname, status
transactions	accountId → accounts, type, amount, balanceAfter, relatedAccount, flagged, createdAt
approvals	accountId → accounts, staffId → users, decision, remarks

- users.email is unique and indexed for login lookup.
- accounts.userId supports common user-to-account queries.
- beneficiaries.accountId supports owner-beneficiary lookup.
- transactions.accountId is indexed with createdAt for statement/ledger queries.
- approvals.accountId is indexed for approval history.
- Account number and beneficiary ownership combinations have uniqueness constraints.

ER-style relationship: USER 1:N ACCOUNT; USER 1:N APPROVAL; ACCOUNT 1:N BENEFICIARY; ACCOUNT 1:N TRANSACTION; ACCOUNT 1:N APPROVAL.

6. Business Rules, Security & Error Handling

- Passwords are stored as bcrypt hashes; plain-text passwords are never persisted.
- Login issues a JWT. Protected routes require Bearer authentication.
- Role middleware restricts staff/admin operations.
- Customers can access only accounts they own.
- New accounts begin as pending and require staff approval before becoming active.
- Transfers require active source/destination accounts and a trusted beneficiary for external transfers.
- Savings accounts enforce a configurable minimum balance.
- Daily transfer amount is limited by a configurable account limit.
- Large transactions are automatically marked flagged for staff review.
- Transfer debit and credit are committed together through a MongoDB transaction.
- Frozen accounts cannot perform normal transactions.
- Validation is performed by Express middleware before business logic.
- Centralized error handling returns consistent JSON with an errorCode.

Configuration

All secrets and environment-specific values are stored in .env. The repository contains .env.example, while .env is excluded through .gitignore.

7. API Demonstration

Screenshot placeholders – replace with your own Postman results:

- Register → 201 Customer registered and account application created.
- Login → 200 with JWT.
- Staff approval → 200 with Approved status.
- Transfer → 201 with transfer reference.
- Statement → 200 with date-range transactions.
- Flagged transactions → staff monitoring result.
- Freeze/Unfreeze → 200 status updates.

[INSERT POSTMAN SCREENSHOTS HERE]

8. Testing & Results

Test	Expected result
Valid registration	201
Duplicate email	409
Invalid required field	400
No JWT on protected route	401
Wrong role on staff route	403
Approve pending account	200
Insufficient balance/minimum balance conflict	409
Daily limit exceeded	409
Untrusted beneficiary transfer	403
Valid transfer	201
Suspicious threshold transaction	Flagged
Invalid MongoDB ID	400
Freeze account	200
Transaction while frozen	409
Interest calculation	Interest ledger entry

Testing evidence: Add screenshots of successful and failure Postman responses in this section.

9. GitHub, Challenges, Learnings & Conclusion

GitHub Repository: <PASTE GITHUB LINK HERE>

GitHub hygiene

- Repository should contain source code, README, Postman collection and .env.example.
- node_modules and .env must not be committed.
- Commit history should show meaningful contributions from multiple team members.

Challenges and Solutions

- MongoDB transfer consistency → used a MongoDB session transaction for source debit and destination credit.
- Business-rule enforcement → placed validation and rule checks in backend controllers/middleware rather than relying on a frontend.
- Role security → centralized JWT authentication and role authorization middleware.
- Monitoring → indexed transaction/account fields used by staff dashboard queries.

Conclusion

P10 demonstrates a structured Node.js/Express backend for digital banking operations. The implementation covers all 13 required modules, MongoDB modelling, JWT/bcrypt security, validation, centralized error handling, transaction workflow logic and Postman-based testing.

Submission reminder: add real team details, GitHub link and your own code/Postman/output screenshots before LMS upload.